QuAsm: A Quantum Simulator DSL in Hand-Written RISC-V Assembly, and What It Took to Trust It

Shriharshith Keshav
shriharshithkeshav@gmail.com

October 24, 2026

Abstract

We present QuAsm, a domain-specific language for statevector quantum simulation whose interpreter is written entirely in hand-coded RISC-V assembly. The parser, the complex arithmetic, the symbol tables, the pseudorandom number generator, a depolarising noise channel and all control flow are assembly, with no library underneath. Programs declare registers of up to six qubits, apply single- and two-qubit gates from a universal set including a controlled-phase family sufficient for the quantum Fourier transform, measure and collapse, branch on measurement outcomes, and loop.

The artifact is small; the interesting part is what establishing confidence in it required. We describe six verification layers that fail independently: cross-validation against an independently authored simulator, differential testing against a NumPy oracle deliberately built on different algorithms, a static audit of the gate matrices parsed out of the assembly source, metamorphic properties over random states, a replay scheme that validates stochastic measurement without reimplementing the generator, and grammar-based differential fuzzing. We report three defects the behavioural suite of the time did not detect — a controlled-Z matrix that was unitary but wrong, a $\sqrt{X}$ that was not unitary at all, and a measurement routine that turned a fair coin into a deterministic one — and argue that each was invisible for the same reason: a behavioural test only distinguishes two implementations where its inputs put weight exactly where they differ.

We then measure that claim. The full corpus kills all 628 meaningful seeded mutations of the gate constants; keeping every program but starting every register in $|0\ldots0\rangle$ lowers that to 476, the inputs being the only change, while the static audit stays at 100%. All 150 code mutants of the gate kernel and measurement routine were run against the whole corpus: 146 were killed, the four survivors sat in how a measurement statement is parsed and in one sampling branch, and five targeted tests now kill those too. A second round of hardening — hostile-input probing and the fuzzer — found seventeen further defects in the interpreter and thirteen in the oracle, the test harness and the mutation tool, two of which had deflated a result we previously reported. None was reachable from the 158 hand-written tests, and every one sat in a region of the input space those tests never entered. Written from the grammar alone, the fuzzer re-finds thirteen of the seventeen interpreter defect classes on the unfixed code, and finds no disagreement in 2,900 programs on the fixed code. Agreement with Qiskit over 5,000 random circuits is exact to 8×10^{-16}, and to one float32 ULP through the assembly itself. We close by stating plainly what this system is not: it holds the statevector in the processor's own memory, which is precisely what a quantum control processor does not do.

1 Introduction

Quantum circuit simulators are usually written in Python or C++ on top of a linear-algebra library, and the interesting engineering is in scaling: distributing state, exploiting sparsity, fusing gates. This paper reports on the opposite corner of the design space. In QuAsm, a register of n qubits is 2^n pairs of 32-bit floats in a fixed `.data` arena, a gate is a hand-written loop over amplitude indices, and the interpreter that ties them together — lexer, parser, symbol tables, control flow — is also assembly. There are roughly 3,600 lines of it (3,050 excluding blanks and comments), across fourteen files. Nothing is generated and nothing is linked.

The motivation is pedagogical. QuAsm began as a Computer Organization course project, and a system in which a student can read every instruction between typing `qubitop1 q[0] = H` and seeing an amplitude change is a useful object. But an artifact is only useful if it is right, and the process of making this one right turned out to be more interesting than the artifact. Hand-written assembly implementing numerical kernels is an unusually hostile target for testing: there are no types, no bounds checks, no exceptions, and — critically — the data the kernels operate on is embedded as literal constants in the source, where a single mistyped digit produces a program that runs perfectly and computes the wrong physics.

This paper makes three contributions.

1. **The artifact.** A complete, documented quantum simulation DSL in hand-written RISC-V assembly, with measurement-conditioned real-time control flow, a depolarising noise channel, and a universal gate set (Sections 3–4).

2. **A verification method for this class of code**, in six layers that fail independently — external cross-validation against Qiskit, a deliberately divergent oracle, a static audit of constant data, metamorphic properties, replay for stochastic behaviour, and grammar-based differential fuzzing — together with an argument for why each is necessary and an explicit account of what each cannot establish (Section 5).

3. **Three defect case studies, and a measurement of the structure they share** (Sections 6–8). Each defect was invisible to a behavioural suite that was, by ordinary standards, thorough. Rather than leave that as an anecdote, we seed 702 mutations of the gate constants and show that changing nothing but the starting states of the test programs, to $|0 \ldots 0\rangle$, lowers the suite's kill rate from 100% to 75.8% while leaving the static audit unaffected.

4. **A second round of thirty defects, and the same structure again.** Probing with hostile inputs and fuzzing found seventeen more interpreter defects — none a crash, all silent wrong behaviour — and thirteen in the oracle, the harness and the mutation tool, two of which had understated a result we previously reported and which we retract here (Section 7). Every one lay where the hand-written tests placed no weight: numbers outside the ranges they used, layouts they never wrote, degenerate registers, and text that never runs.

We are deliberate about what we do not claim. QuAsm is not a fast simulator, not a quantum ISA, and not quantum control hardware. Section 9 says so in detail, because the gap between "a classical processor running quantum simulation" and "a classical processor controlling a quantum computer" is the thing most likely to be misread, and it is a real and instructive gap.

2 Background and related work

Statevector simulation. The pure state of n qubits is a unit vector in $\mathbb{C}^{2^n}$; a gate on a subset of qubits is a unitary acting on the corresponding tensor factors, and measurement in the computational basis projects onto a subspace and renormalises [26]. Production simulators — Qiskit Aer [29], QuEST [16], Intel-QS [11] and `qsim` [30] — implement this with vectorised kernels, gate fusion and distribution. QuAsm implements the textbook algorithm directly and caps n at six; it is not competitive and does not try to be.

Quantum languages. Higher-level quantum languages — Quipper [10], Q# [34], Silq [2], Scaffold [14], ProjectQ [33] and Cirq [4] — are hosted or standalone languages that compile circuits for simulators and hardware. QuAsm is far smaller than any of them and aims at something different: not expressiveness, but an interpreter a student can read end to end.

Quantum assembly languages and control processors. OpenQASM 2 [7] could express circuits and a single conditional on a classical register; OpenQASM 3 [6] added real-time classical control precisely because hardware requires it — mid-circuit measurement, feed-forward, and branching within the coherence window. eQASM [9] and the QuMA microarchitecture take this further, defining an executable instruction set with explicit timing for a superconducting device.

There is active work on RISC-V specifically as the classical half of a quantum control stack. Most directly, Liu et al. [21] present a generator for quantum control system-on-chips compatible with RISC-V. *That system and this one share nothing but an adjacency of subject matter*: theirs generates control hardware in SpinalHDL with nanosecond timing guarantees, and ours is a software simulator hosted on a teaching emulator. We note the overlap explicitly to prevent confusion, and we discuss the real architectural distance in Section 9.

Simulation environments and education. QuAsm runs under RARS [19], an open-source RISC-V assembler and runtime simulator widely used in teaching, which supplies I/O through an `ecall` interface; it stands in the same relation to RISC-V as MARS [36] does to MIPS. Quantum computing has been taught through practical software-driven approaches [25] and analysed as a curriculum topic in its own right [31]. In QuAsm the quantum content and the architecture exercise are the same thing: the assembly *is* the simulator.

Differential, metamorphic and mutation testing. Testing one implementation against an independent one is long-established practice. McKeeman [22] named it; Csmith [38] and EMI [20] made it the standard way to find compiler bugs, by generating programs whose correct behaviour is known by construction. Where no oracle exists at all — the usual situation in scientific computing — metamorphic testing substitutes relations between runs for absolute expected values [3, 32], and this is the dominant recommendation in the literature on testing numerical and scientific software [12, 17]. Property-based testing [5] supplies the generator-plus-invariant machinery we use in Section 5.

Mutation testing [8, 15] is how we measure whether any of this works: seed defects deliberately, and count what the suite catches. Whether a test catches a seeded defect at all is governed by the conditions of the propagation-infection-execution model [35]: the defect must be executed, must infect the program state, and the infection must propagate to an output. The mechanism we measure in Section 8 is the infection condition, applied to constant data. Muskit [23] mutates *gates in a Qiskit circuit* — the program under test — to assess a test suite for quantum programs [24, 1].

```
qubit q[2]                      # 2-qubit register, initialised to |00>
qubitop1 q[0] = H               # superposition
qubitop2 q[0],q[1] = CN         # entangle: a Bell pair
print(q)                        # [(0.70710677,0.0),(0,0),(0,0),(0.70710677,0.0)]
measure q[0] -> c               # collapse; keep the outcome in c
if c == 1                       # undo the flip, whichever way it landed
qubitop1 q[0] = X
qubitop1 q[1] = X
endif
print(q)                        # always |00>
end
```

Figure 1: A Bell pair, a mid-circuit measurement, and a correction conditioned on its outcome. The `-> c` tail is what makes the result available to `if`.

We mutate the *constants and the kernel of the simulator itself*, one level down: our subject is the implementation that Muskit's subject would run on. That level has been tested before. QDiff [37] differentially tests quantum software stacks against one another, MorphQ [28] applies metamorphic testing to Qiskit, and Paltenghi and Pradel [27] study 223 real bugs in eighteen quantum computing platforms, most of them not quantum-specific. Those works find bugs in production platforms through their public interfaces; what we add is a measurement, on a simulator small enough to audit its constant data directly, of how much of one defect class a behavioural suite reaches.

The contribution here is therefore not the idea of differential testing but three things that follow from applying it to hand-written assembly: a concrete account of what "independent" has to mean for it to have teeth (Section 5), a failure mode — wrong constant data — that differential testing on its own does not address (Section 6), and a measurement of exactly how much of that failure mode a behavioural suite reaches (Section 8).

3 The language

A QuAsm program is a sequence of statements, one per line, ending in `end`. There are no expressions, no functions, and two types: qubit register and classical integer. Figure 1 shows the language in use.

Registers and gates. `qubit v[n]` declares an n-qubit register ($1 \le n \le 6$) in $|0\ldots0\rangle$; `qubit v = [(re,im),...]` declares one from explicit amplitudes. A register of n qubits is 2^n contiguous amplitudes in basis order $|q_0 q_1 \ldots q_{n-1}\rangle$, so qubit 0 is the *most* significant bit of an amplitude index. (Qiskit uses the opposite convention; this trips people up, and the documentation says so.) Single-qubit gates are H, I, X, Y, Z, S, T, $\sqrt{X}$ and the inverses $S^\dagger$ and $T^\dagger$; two-qubit gates are CNOT, SWAP, controlled-Y, controlled-Z, and a controlled-phase family

$$\mathrm{CP}k = \mathrm{diag}\left(1,\, 1,\, 1,\, e^{2\pi i/2^k}\right), \qquad k = 1\ldots6.$$

Since $\{H, T, \mathrm{CNOT}\}$ is universal, any unitary is in principle expressible. The CP family exists for a more immediate reason: it is what the quantum Fourier transform needs, and $k \le 6$ is as fine a rotation as a six-qubit register can use. Two-qubit matrices are written in the $|\text{control target}\rangle$ basis, so `qubitop2 q[1],q[0] = CN` is a genuinely different operation from `qubitop2 q[0],q[1] = CN` and both are tested.

Measurement and real-time control. `measure v[k]` projects one qubit, prints $|0\rangle$ or $|1\rangle$, and renormalises; `measure v` samples a basis state from the whole distribution and collapses onto it. Either may end `-> c`, which keeps the outcome as an integer in a classical variable. Together with `if`/`else if`/`else`/`endif` and `repeat`/`endrepeat`, that is enough for conditional correction, repeat-until-success loops, and the classical half of a teleportation-style protocol. This is the same capability OpenQASM 3 added for hardware, in miniature: a branch whose condition is a measurement outcome produced earlier in the same program.

Input. Integers are 32-bit, and decimals — which may carry an exponent — must be zero or of magnitude in $[10^{-18}, 10^{18})$, so that the squared amplitudes of a full register can neither overflow nor underflow float32 when summed. A line holds at most 1022 bytes. Anything outside those limits is an error, never a wrapped or truncated value, and every error names its line. Section 7 explains why each of those sentences had to be written.

Noise. `noise p` enables a depolarising channel: after every gate, each qubit that gate touched has probability p of receiving a uniformly chosen Pauli. This is the quantum-trajectory form — one run is one trajectory, and the channel is what the ensemble converges to. Section 8 shows the resulting fidelity decay.

4 Implementation

Memory. Everything is fixed-size `.data`: a 4096-byte amplitude arena (bump-allocated, eight maximum-size registers), a 1000-byte register symbol table, a 256-byte classical variable table, a 2048-byte store for recorded loop bodies, and a 1024-byte input buffer. There is no allocator. Every limit in the language is a number visible in the source.

The gate kernel. Two routines, both acting in place, both taking arguments in registers rather than through shared globals. `apply_gate1` walks all 2^n amplitude indices and, for each pair differing only in the target bit, applies the 2×2 matrix:

$$(s_i,\ s_j) \leftarrow (u_{00}s_i + u_{01}s_j,\ u_{10}s_i + u_{11}s_j), \qquad j = i \,|\, \mathrm{bit}(k),$$

computing both results before storing either. `apply_gate2` does the same for the groups of four indices differing in the control and target bits: gather, apply, scatter. Complex multiply-add is inlined throughout. Qubit k occupies bit $n-1-k$, a line that appears identically in the kernel, in measurement, and in the oracle.

Measurement. `rand_u32` is xorshift32, seeded to a fixed nonzero constant so that a session is reproducible by default. `rand_unit` takes the top 24 bits — xorshift's low bits are its weak ones, and 24 is exactly float32's mantissa width, so both numerator and 2^{24} are exact. Single-qubit measurement sums the weight of both branches, draws r, and selects outcome 1 iff $r \cdot \mathrm{total} < P(1)$; Section 6 explains why both branches are summed rather than one inferred from the other. A guard rejects a selected branch whose probability is below 10^{-9} as float32 noise rather than a real outcome, since $1/\sqrt{p}$ would otherwise destroy the register.

Control flow without seeking. The interpreter is not file-aware: a script is piped to standard input exactly as an interactive session is, and standard input cannot be rewound. A loop body therefore has to be remembered. Lines are copied into a store *only while a loop is open*, and the store is dropped when the outermost loop closes, so straight-line programs of any length never touch it. A frame stack of (body address, iterations remaining) drives replay, where `repeat` pushes a frame whose body address is "wherever the next line is going to come from" — which makes nesting work with no second mechanism.

Four situations require lines to be read but not executed: the body of a `repeat 0`, an untaken `if` branch, the rest of the loops after the store overflows, and the rest of the program after an `end` inside a block. One mechanism covers all four. It keeps a small block stack — one bit per open block for its kind and one for whether it has had its plain `else` — that tracks skipped blocks as well as running ones, so that text which never runs is still held to the rules of block structure, as a parser that saw the whole program would hold it. The observation that lets a single `(kind, base)` pair describe the skip itself is that *a skip can never begin while one is already in progress*, because the statement that would start it is never executed while skipping.

Because branches are decided line by line on the stream rather than by seeking, an `if` inside a loop body is re-evaluated on every iteration for free: the recorded lines replay and the condition is read again against whatever the classical variables now hold.

5 Verification

The suite is 230 programs, run through both the assembly under RARS and an independent oracle; an expected error must be an error to both, and a program not expected to fail may report no error at all. Layers 1–3 did the original work. Layers 0, 4 and 5 were added afterwards, in response to the weakness the first three share — every one of them was written by the same person who wrote the implementation — and are numbered to keep the original three recognisable, since Section 6 refers to them by number. They are presented here in the order a defect meets them.

Layer 0: cross-validation against an independently authored simulator. Layers 1–3 are all ours. If a misconception about the semantics is shared between the assembly and the oracle, they agree and say nothing — and being written in a different language with different algorithms does not protect against a misconception, only against a transcription error. `cross_validate.py` therefore compares against Qiskit [29], which no one here wrote.

Agreement is real evidence about gate semantics and the kernel, but it is bounded in a way the paper should state plainly: the *mapping* between the two projects' conventions is ours, and a wrong mapping quietly adjusted until the disagreements vanished would manufacture false agreement. Two rules prevent that. First, the mapping is fixed a priori from the two projects' documented conventions — QuAsm's qubit 0 is the most significant bit of an amplitude index, Qiskit's is the least, so the mapping is the reversal $k \mapsto n - 1 - k$ and nothing else — and is never adjusted in response to a failing comparison. Second, the mapping is itself tested by circuits whose output is asymmetric under qubit permutation: X on qubit 0 alone of a three-qubit register must give $|100\rangle$, and an inverted mapping fails such tests loudly rather than hiding inside an averaged agreement. Deliberately inverting the mapping is caught by all five asymmetric tests; deliberately corrupting a gate constant is caught on every circuit that uses it.

What this does *not* establish is that a mapping error symmetric in exactly the way both sides are wrong would be caught. We claim no more than that.

Layer 1: a deliberately divergent oracle. `reference_sim.py` implements the language semantics in NumPy with `complex128`. Being "independent" is not achieved by being written in a different language — it is achieved by using different algorithms, so that a shared misconception is less likely to cancel:

- Where the assembly walks amplitude indices bit by bit, the oracle reshapes the state to an n-dimensional $(2, 2, \ldots, 2)$ tensor and contracts with `tensordot`/`moveaxis`.
- Where the assembly executes control flow by recording lines and replaying them against a frame stack, the oracle parses a real block tree and walks it recursively.

Agreement between two implementations that reach the same answer by different routes is evidence. Agreement between an implementation and its transcription is not.

Layer 2: static audit of constant data. A behavioural test distinguishes two matrices only where its input state has nonzero amplitude at a position where they differ. Wrong *data* can therefore pass an arbitrarily large behavioural suite whose inputs all miss that position — and size is no protection, since it is a property of where the inputs place weight, not of how many there are (Section 8 measures this). `check_gate_data.py` parses the `.float` declarations directly out of the assembly source, reconstructs each matrix, and checks that it is unitary and equal to the gate it claims to be. It runs as a pre-flight on every test invocation and covers all 20 matrices. Section 6 describes the two defects it found after the behavioural suite was already green.

Layer 3: replay for stochastic behaviour. Measurement cannot be compared value-for-value, since the two implementations draw from different streams, and reimplementing xorshift32 in Python would convert the oracle into the transcription check Layer 1 exists to avoid. Instead the harness *replays the outcomes the assembly actually reported* and checks three independent properties:

1. **Possibility.** Every reported outcome must have had probability $> 10^{-6}$ under the oracle's exact arithmetic. This is sharper than it sounds: in one test only 2 of 8 outcomes are possible, in another only 4 of 64, and several leave exactly one legal outcome for a later measurement.
2. **Collapse.** The state printed after a measurement must equal the collapse-and-renormalise that outcome implies, to 10^{-4}.
3. **Frequency.** Empirical frequencies over many trials must match the Born-rule probability within a stated tolerance (≈ 3.3 standard errors).

Both directions of this check were confirmed to fire, by feeding the comparator a wrong amplitude vector and an impossible outcome.

Noisy programs are a fourth case: a trajectory depends on which Paulis the assembly's own generator drew, so there is no state to diff. Those tests are judged on statistics alone, and the harness refuses a noisy program that does not supply a statistical expectation.

Layer 4: metamorphic and property-based tests. A behavioural test fixes an input and an expected output. A property fixes a *relation* that must hold for every input, and is then checked over a seeded random sample. `properties.py` implements eight. Five are: norm preservation under every gate, including on registers that are deliberately not normalised; the involution and inverse

identities ($XX = I$, $\mathrm{SN}^2 = X$, $S^2 = Z$, $T^2 = S$, $\mathrm{CN}^2 = I$) on random asymmetric states rather than fixed ones; permutation equivariance, expressed inside the language as $\mathrm{SW}(i,j)\,G(j)\,\mathrm{SW}(i,j) = G(i)$; collapse invariants, that a measured register is left normalised and the reported outcome was possible; and Born-rule agreement between the single-qubit and whole-register measurement forms *on unnormalised registers*.

The Born-rule property is the third defect stated as an invariant, and it is the one that matters most here: it is the only check in this entire system that would have caught a defect all three original layers missed. Each property is validated by deliberately violating its invariant and confirming it fails — scaling $\sqrt{X}$ by $\sqrt{2}$, putting T a hundredth of a radian out, replacing SWAP with the identity, skipping renormalisation, and reinstating the original $P(0) = 1 - P(1)$ shortcut. Three more relate different gates rather than a gate to its inverse — circuit identities such as $\mathrm{SW} = \mathrm{CN}\,\mathrm{CN}\,\mathrm{CN}$ and $H\,\mathrm{CZ}\,H = \mathrm{CN}$, the 15-gate Clifford+$T$ Toffoli checked against the permutation it must perform, and $\mathrm{QFT}^4 = I$ — and are validated the same way.

Layer 5: differential fuzzing. Every layer above checks cases someone chose. `fuzz.py` generates them, from the grammar rather than from any list of known defects: well-formed programs with registers declared by size or by explicit amplitudes, every gate, prints, measurements with and without a classical target, conditionals on measured values and nested loops, laid out with random indentation and spacing; and those same programs with one line corrupted by a random edit — a number made enormous or malformed, a keyword, name or gate replaced, whitespace inserted, a block keyword duplicated or deleted, a line padded past the buffer. A well-formed program is judged as a test program is, with the oracle replaying the outcomes RARS reported, since measurement decides which branches run. A corrupted one is judged first on a single question: do the two implementations agree that it is an error? Runaway programs, which the two sides bound differently, and the three limits of the assembly's fixed memory that the oracle does not model are deliberately not findings.

6 Three defects, and why they were invisible

A unitary, wrong controlled-Z. The shipped matrix was $\mathrm{diag}(1, 1, -1, -1)$, which is $Z \otimes I$, not $\mathrm{diag}(1, 1, 1, -1)$. It is perfectly unitary, so nothing misbehaved. It survived four dedicated controlled-Z tests because all four shared an input state whose amplitude at $|10\rangle$ was zero — exactly and only where the two matrices differ. Found by Layer 2 in the first run of the static audit.

A non-unitary $\sqrt{X}$. The $\sqrt{X}$ matrix carried a 0.707 normalisation, giving column $\mathrm{norm}^2 = 2$. It was not a unitary at all — it did not preserve probability — and the behavioural suite had not noticed. The matrix is otherwise exactly $\sqrt{X}$, which needs 0.5. Also found by Layer 2.

A deterministic fair coin. Single-qubit measurement computed $P(1)$ by summing $|a_i|^2$ over indices with the target bit set, then took $P(0) = 1 - P(1)$. That shortcut is valid only for a normalised register, and the language deliberately permits declaring one that is not. Given `qubit a = [(2,0),(2,0)]` — a fair coin up to normalisation — the routine computed $P(1) = 4$, so $r < P(1)$ held for every $r \in [0, 1)$ and the measurement was *deterministic*. Across 200 trials it returned $|1\rangle$ 200 times. The whole-register form failed differently rather than identically: on the unnormalised two-qubit state `[(0.6,0),(0,0),(0,0),(0.6,0)]` — also a fair coin up to normalisation — it returned 85/115 where 100/100 was correct, because its cumulative-sum walk skews rather than saturates.

The two measurement forms therefore disagreed with each other on states of the same kind, which is what makes the defect a semantic inconsistency and not merely a missing normalisation.

This third defect is the most instructive, because *all three verification layers missed it.* Layer 2 does not apply: no constant was wrong. Layer 1 did not fire, because the oracle computes both sums correctly and therefore agreed with the assembly on the *collapsed* state — the only value being compared. Layer 3's possibility check passed, because the oracle assigned the reported outcome probability 4, comfortably above the threshold. Only a frequency check on an unnormalised register would have caught it, and no test used one.

The fix is to sum both branch weights and scale r by their total, costing one addition and one multiplication per measurement, and making the two measurement forms agree. Three regression tests now cover it.

The common structure is worth stating: **each defect was a case where the test inputs did not put weight where the correct and incorrect behaviours differ**. For matrices, that is a zero amplitude at the distinguishing index. For the measurement bug, it is a normalisation invariant that every test happened to satisfy. In all three cases the offending code was executed many times over by the suite — execution was never the problem.

The lesson we draw is that for numerical code the useful question is not "is this path executed?" but "does any test place mass where a plausible wrong version would differ?", and that the second question is not answered by any coverage metric we know of. That scepticism is not novel — Inozemtseva and Holmes [13] found coverage only weakly correlated with test suite effectiveness once suite size is controlled for — but the mechanism here is unusually legible, because the distinguishing index can be pointed at.

Section 8 turns this into a measurement rather than a moral. Restricting the Keeping every test program but starting each register in $|0\ldots0\rangle$ — removing precisely the property that places weight at the distinguishing indices, and nothing else — lowers the fraction of seeded constant-data defects the suite detects from 100% to 75.8%, while leaving the static audit at 100%. The recommendation that follows is concrete and cheap: for numerical kernels, audit the constants directly, and make sure some input puts nonzero weight everywhere the data could be wrong.

7 Seventeen more, and the same structure

The three defects above were found by building verification layers. The next round was found by attacking the inputs: forty-five hostile programs written by hand, and then the fuzzer of Layer 5. They found seventeen more defect classes in the interpreter. None crashed it. Every one produced a silently wrong result, and all 158 hand-written tests passed with every one of them present.

- **Numbers.** Integer literals wrapped modulo 2^{32}, so `q[4294967297]` addressed qubit 1 and `repeat 4294967296` ran no times. The decimal parser accumulated fractional digits into a 32-bit integer, so `0.60000000000000000000000001` was read as 2.4285715.

- **Degenerate registers.** Measuring an all-zero register produced NaN in the single-qubit form and silently collapsed to $|00\rangle$ in the whole-register form — the two forms disagreeing, exactly as in the third defect. The guard against rounding noise was an absolute 10^{-9}, so a register of total weight 10^{-10} reported an outcome of probability zero and then divided by it.

- **Layout.** Tabs, indentation, spaces inside brackets and a comment directly after a keyword were all syntax errors, and a line longer than the 100-byte buffer was cut short without a word: `qubitop1 q[0] = X`, padded, then `Y`, ran as `X`.

- **Structure.** A second `else` was accepted, a closer of the wrong kind was accepted, nothing in skipped text was checked, a block left open at `end` was never reported, and a loop too long for its store, inside an `if`, took the rest of the program with it.

The oracle had seven defects of its own — three crashes, three things it accepted that it should not, and a runaway guard that counted statements, so an empty loop body with a large count spun for hours — and the harness four: it never checked that the oracle agreed a program was an error, never failed a test that printed an unexpected error, judged an outcome's possibility on an unnormalised weight, and compared with an absolute tolerance values that float32 holds only relatively. Two more were in the mutation tool, and they matter because they understated a result this paper previously reported (Section 8).

The structure is the one Section 6 described, one level up. There, the inputs never placed weight at the matrix entries where a wrong implementation differed; here, the corpus never placed weight in the regions of the input space where the implementation was wrong — numbers beyond the ranges the tests happened to use, layouts nobody wrote, registers with no weight, error paths in text that never runs. A corpus written by the author of the code shares the author's sense of what an input looks like. The fuzzer does not, which is why it re-found most of these on the unfixed code (Section 8); and the few it missed sit, again, where its own generator places no weight.

8 Evaluation

Correctness. All 230 programs pass against both the assembly and the oracle, and all 20 gate matrices pass the static audit. The suite covers each single- and two-qubit gate on inputs with all amplitudes distinct and nonzero, gate identities by composition (HH, XX, $SS = Z$, $TT = S$, $\sqrt{X}^2 = X$, $HXH = Z$), multi-qubit states (Bell, GHZ-3, GHZ-4, CNOT across non-adjacent qubits, a six-qubit maximum register), declaration and aliasing edges, measurement and collapse, statistical checks, loops, conditionals, circuits with known answers (the Clifford+T Toffoli on every basis state, $\mathrm{QFT}^4 = I$, Bernstein–Vazirani, Grover, superdense coding and teleportation), a six-qubit register with all 64 amplitudes given, and 85 error paths.

External cross-validation. Table 1 reports agreement with Qiskit over seeded corpora of random unitary circuits. Every register is declared with explicit random amplitudes, all distinct and nonzero, on every register size. Every corpus is regenerable byte for byte from its seed, and the tool prints a SHA-256 over the generated programs so a reviewer can confirm they have the same corpus.

Agreement is exact to the limit of the arithmetic. Against the oracle, where both sides compute in `complex128`, the worst elementwise deviation over 5,000 circuits is 8.0×10^{-16} — machine epsilon. Against the assembly, where one side is float32, it is 1.5×10^{-7}, about one ULP, and three orders of magnitude inside the 10^{-4} comparison tolerance. There were no disagreements of any kind.

Mutation testing: how much of the constant-data failure mode does a behavioural suite reach? Section 6 argues that behavioural tests reach wrong constant data only where their inputs place weight. That argument rests on three anecdotes; this section measures it.

We generate 702 mutants of the gate matrices in `main.s` by five operators — sign flip, zeroed entry, single-digit change, entry swap, and a one-ULP float32 nudge — and classify each as unitary-preserving or not, since only the unitary-preserving class is genuinely hard (a unitarity check catches the rest by construction). Each mutant is screened by substituting the mutated matrix into the

comparison	seed	circuits	qubits	worst deviation
oracle vs. Qiskit	1	5,000	1–5	8.01×10^{-16}
oracle vs. Qiskit	0	500	1–4	4.97×10^{-16}
Qiskit vs. QuAsm (RARS)	0	200	1–4	1.53×10^{-7}

Table 1: Cross-validation against Qiskit 1.3.1. Each row is regenerable with `cross_validate.py --seed S --n N`, which also prints a SHA-256 over the generated corpus. Zero disagreements throughout; the minimum overlap $|\langle\psi_a|\psi_b\rangle|$ was 1.000000000000 to twelve places in the exact comparison and 0.999999999999997 against float32. Deliberately inverting the qubit mapping is caught by all five asymmetric mapping tests, and deliberately corrupting a gate constant is caught on every circuit that uses that gate.

oracle and re-running every test program that applies that gate; a mutation to gate G cannot change the behaviour of a program that never applies G, so this restriction is sound rather than a sampling shortcut. A mutant the screen kills by a wide margin — a deviation of at least 10^{-2}, a hundred times the tolerance — needs no confirmation, since float32 rounding cannot rescue it. Every other mutant, every survivor and every narrow kill included, is rebuilt as a genuinely mutated `main.s` and run through the real harness under RARS.

Against the full corpus, the behavioural suite kills every meaningful constant mutant: 628 of 628, the 74 one-ULP nudges being excluded because they sit below the comparison tolerance by construction. That is not an accident of the operators. After the first two defects were found the corpus was deliberately extended to exercise every gate on inputs with all amplitudes distinct and nonzero, which is exactly the property that makes a wrong entry visible. The forty-program suite that shipped the defects passed green with both present; we ran that experiment at the time, but the revision history it needs is not part of the artifact, so we report it as history rather than offer it for re-running.

To isolate the mechanism we change one thing. *Keeping every program and every statement, but starting every register in* $|0\ldots0\rangle$ *instead of the state it declares*, lowers the kill rate from 628 to 476 of 628 (75.8%), and the hard, unitary-preserving class from 124 to 94 (Table 2). The static audit kills every one of them in every condition, because it does not depend on test inputs at all. That is the entire argument for having it.

An earlier version of this paper reported a stronger-looking result: dropping the programs that declare such states halved the kill rate. That experiment changes which programs run as well as what they start from, and on the same corpus it removes more kills (382 of 628 remain) than the change of inputs alone. On the original 158-program corpus, the one that earlier version measured, the change of inputs alone lowers the kill rate from 594 of 594 to 426, and dropping the programs lowers it to 302: of the 292 kills the reported halving removed, 168 are attributable to the inputs and the rest to the programs that went with them. We now report the controlled figure as the measurement of the mechanism, and the larger one only as an upper bound that folds in the loss of the programs themselves.

Mutating the code, not the data. The second class touches the gate kernel and the measurement routine rather than the constants: branch conditions flipped, immediates moved by one, and operands swapped on non-commutative instructions — all 150 such mutants, each run under RARS against the whole corpus, error tests included. The suite as it stood kills 146 of the 150 (97.3%). None of the four survivors is in the gate kernel. Three are in how `measure` reads its target — the arrow and the one-letter name — and each changes behaviour only on a spelling no test used, such

		behavioural suite			static
mutant class	total	full corpus	inputs $\|0\ldots0\rangle$	those programs only	audit
constant, unitary-preserving	124	124	94	73	124
constant, non-unitary	504	504	382	309	504
all, excluding ULP nudges	628	628	476	382	628

Table 2: Constant mutants killed, by layer and by corpus. *Inputs* $|0\ldots0\rangle$ keeps every program and statement and changes only the state each register starts in; *those programs only* drops the programs that declare any other state. The 74 one-ULP nudges are below the comparison tolerance by construction and are excluded throughout.

as `q[0]->c` with no spaces; the fourth is a branch in whole-register sampling that still draws the right distribution when a register has two equally likely outcomes, and only then — which is what every whole-register frequency test in the corpus happened to measure.

We retract what an earlier version of this paper said here. It reported that the suite killed 25 of 30 sampled code mutants and that every survivor lay in the measurement routine, and read that as mutation analysis and defect history pointing independently at the same file. The harness behind that number ran each code mutant against only the first 25 programs in file order — none of them a measurement test — and skipped every error test. The kernel has no surviving mutant; what survives is how a measurement statement is parsed, and one branch of whole-register sampling whose effect shows only in statistics the corpus never gathered. Five targeted tests, now in the corpus, kill all four, which brings the code mutants to 150 of 150.

Two mutant classes are excluded as equivalent rather than counted as survivors, because nothing could kill them and including them would flatter the argument this section is testing: the one-ULP nudges above, and operand swaps on commutative instructions (`fadd.s`, `fmul.s` and their integer counterparts), where the mutation is semantics-preserving by construction.

Fuzzing. Table 3 reports both directions. On the fixed code, 2,900 generated programs — half well-formed, half corrupted — produce no disagreement. The runs raised two on the way, and both were in our harness rather than in either implementation: the judge had let the oracle draw its own measurement outcomes, so the two sides took different branches, and it compared an amplitude of 10^{11} — legal — with an absolute tolerance that float32 cannot meet at that magnitude. Pointed at the code as it stood before this round, with the generator restricted to the language as then documented, 2,600 programs produce 133 disagreements. By hand we assign them to thirteen of the seventeen interpreter defect classes of Section 7, plus an oracle crash; the fuzzer also hung, on the oracle's runaway gap. It found none of the other four — the all-zero and tiny registers, the doubled `else`, and the loop too long for its store inside an `if` — all inputs its generator never produces. That is the thesis of this paper applied to its own newest layer, and the reason the fuzzer is one layer among six rather than a replacement for the rest.

The quantum Fourier transform. With the CP family the QFT is directly expressible. On $|001\rangle$ of a three-qubit register the implementation produces amplitudes matching $\frac{1}{\sqrt{8}}e^{2\pi ijk/8}$ to seven decimal places (Table 4); the four-qubit case is likewise verified against the oracle.

code under test	programs	disagreements	interpreter defect classes found
before this round (grammar as then documented)	2,600	133	13 of 17, and an oracle crash
after it	2,900	0	—

Table 3: Differential fuzzing, the assembly against the oracle, each on its own tree. Half of each run is well-formed programs and half is the same programs with one line corrupted. The classification of the 133 into defect classes is by hand.

k	QuAsm (float32)	analytic
0	`(0.35355335, 0.0)`	$(0.35355339,\ 0)$
1	`(0.24999997, 0.24999997)`	$(0.25,\ 0.25)$
2	`(0.0, 0.35355335)`	$(0,\ 0.35355339)$
3	`(-0.24999997, 0.24999997)`	$(-0.25,\ 0.25)$
4	`(-0.35355335, 0.0)`	$(-0.35355339,\ 0)$
5	`(-0.24999997, -0.24999997)`	$(-0.25,\ -0.25)$
6	`(0.0, -0.35355335)`	$(0,\ -0.35355339)$
7	`(0.24999997, -0.24999997)`	$(0.25,\ -0.25)$

Table 4: QFT of $|001\rangle$ on three qubits. Every amplitude agrees with $\frac{1}{\sqrt{8}}e^{2\pi ijk/8}$ to within 5×10^{-8}, roughly one unit in the last place of float32.

Numerical stability. Gate constants are stored to eight decimal places, finer than float32 resolves, so the only error source is rounding across multiply-adds. We measured its accumulation by preparing $|+\rangle$ and then applying H repeatedly in pairs, each pair being an application of $H^2 = I$ (Table 5).

The result is better than a drift analysis would predict: the error does not accumulate at all. The amplitude leaves `0.70710677` — the float32 nearest $1/\sqrt{2}$ — and settles one ULP below it at `0.7071067` after the very first H^2, and then does not move again through two thousand gates. The rounded value is a fixed point of the float32 H^2 map, so the sequence is periodic rather than diffusive, and total error stays bounded at 5.96×10^{-8} regardless of circuit depth. Composed identities such as $T^8 = I$ behave the same way.

This should not be over-generalised — it is a property of this particular map at this particular value, not a guarantee for arbitrary circuits — but it does mean the 10^{-4} comparison tolerance sits three orders of magnitude above observed rounding while remaining thousands of times below the $O(0.1\text{–}1)$ discrepancy a wrong matrix entry produces. The tolerance separates rounding from error by a wide margin in both directions.

Noise. With the depolarising channel at rate p, a qubit in $|0\rangle$ reads 1 after one gate with probability $2p/3$, since X and Y flip it and Z does not; measured over 300 trials at $p = 0.3$ we observe 0.217 against a predicted 0.200 (0.7σ). Each noisy gate shrinks the Bloch z component by $(1-4p/3)$, giving $P(|0\rangle) = (1+(1-4p/3)^N)/2$ after N gates; Table 6 shows the measured decay. A two-qubit gate exposes both its qubits independently, and the resulting joint distribution over $|00\rangle, |01\rangle, |10\rangle, |11\rangle$ matches the independent-flip prediction across all four outcomes.

Instruction cost. RARS reports executed instruction counts only when it halts at a step limit, so exact counts for a terminating program are recovered by binary search on that limit; the cost

extra H gates	amplitude	deviation from float32 $1/\sqrt{2}$
0	`0.70710677`	0
2	`0.7071067`	-1 ULP
20	`0.7071067`	-1 ULP
100	`0.7071067`	-1 ULP
400	`0.7071067`	-1 ULP
1000	`0.7071067`	-1 ULP
2000	`0.7071067`	-1 ULP

Table 5: Float32 error does not accumulate under repeated $H^2 = I$. One ULP (5.96×10^{-8}) is lost on the first application; the value is then a fixed point.

N gates	predicted $P(\|0\rangle)$	observed (300 trials)
2	0.876	0.850
5	0.744	0.690
10	0.620	0.580
20	0.529	0.537

Table 6: Fidelity decay under depolarising noise at $p = 0.1$, against $\left(1 + (1 - 4p/3)^N\right)/2$.

of a gate is the difference between a program applying twenty and one applying none. Both series turn out to be exactly linear in 2^n (Table 7):

$$C_1(n) = 24.5 \cdot 2^n + 645, \qquad C_2(n) = 103.25 \cdot 2^n + 923,$$

fitting all eleven measured points with zero residual — unsurprising for a deterministic emulator executing straight-line loops, but a useful check that the measurement recovers what the code says it should. The coefficient is the kernel, confirming the $\Theta(2^n)$ the loop structure requires, and the $4.2\times$ ratio between them reflects the four complex multiply-accumulates per amplitude in the 4×4 case against two in the 2×2.

The constant term is the more interesting quantity. Parsing one statement costs 645 instructions for a single-qubit gate and 923 for a two-qubit one — independent of n, and the latter larger because it parses two operands. That overhead exceeds the kernel entirely until $n = 5$ for single-qubit gates and $n = 4$ for two-qubit ones: for small registers this is not a physics simulator that also parses, it is a parser that also does physics. Even at the six-qubit maximum, 29% of a single-qubit gate's cost is still parsing. Both constants are larger than an earlier version of this paper reported (349 and 524): the interpreter now normalises every line — tabs, comments, indentation — and checks every number for overflow before it parses anything, and those checks are paid on every statement. The kernel coefficients are unchanged, as they should be, since the kernel is. This is the characteristic shape of an interpreted simulator, and it is the number a compiled implementation of this language would have to beat.

9 What this is not

It would be easy to read "RISC-V processor running quantum operations" as a claim about quantum control hardware. It is not one, and the distance is worth stating precisely.

n	amplitudes	1-qubit gate	per amplitude	2-qubit gate	per amplitude
1	2	694	347.0	—	—
2	4	743	185.8	1336	334.0
3	8	841	105.1	1749	218.6
4	16	1037	64.8	2575	160.9
5	32	1429	44.7	4227	132.1
6	64	2213	34.6	7531	117.7

Table 7: Executed RISC-V instructions per gate, by register size. Both series are exactly linear in 2^n: a single-qubit gate costs $24.5 \cdot 2^n + 645$ instructions and a two-qubit gate $103.25 \cdot 2^n + 923$. These fit all eleven measured points with zero residual. The constant term is the parser; the coefficient is the kernel.

QuAsm holds the statevector in the processor's own memory, which is exactly what a quantum control processor does not do. A real controller never possesses amplitudes; it emits pulses and receives measurement bits. The central data structure here is the one piece of state that physical control hardware structurally cannot have, and every scaling limit in this system — six qubits, a 4096-byte arena — is a consequence of simulating on the controller.

Four further gaps follow:

- **No notion of time.** A control processor is defined by deterministic nanosecond gate issue and microsecond feedback latency. QuAsm has no cycle budget, no schedule and no latency model. Its `if` on a measurement outcome has the *shape* of feed-forward with none of the timing semantics that make feed-forward difficult.
- **Trajectories, not channels.** The noise model samples one Pauli trajectory per run. It reproduces the depolarising channel in expectation but does not track a density matrix, and it models neither T_1/T_2 relaxation, coherent control error, nor readout error.
- **Six qubits.** A pedagogical bound, not a scaling result.
- **Simulator-specific I/O.** Input and output go through RARS `ecall`s, which are neither a hardware nor a Linux ABI, so the system requires RARS. Porting to bare metal means rewriting all I/O.

What *is* fair to say is narrower and, we think, still worth saying: the combination of mid-circuit measurement, a classical variable holding its outcome, and branching on that variable is the real-time control structure that OpenQASM 3 and eQASM introduced because static circuit descriptions could not drive hardware — and it is implementable in a few hundred instructions on a minimal classical host, with the quantum backend abstracted behind two routines that take their arguments in registers. That is a statement about the modest size of the classical control surface, not a claim to have built a controller.

10 Conclusion

QuAsm is a quantum simulation DSL whose interpreter is about 3,600 lines of hand-written RISC-V assembly, verified in six layers: cross-validation against Qiskit, differential testing against a deliberately divergent oracle, static auditing of its constant data, metamorphic properties over

random states, replay-based checking of its stochastic behaviour, and grammar-based differential fuzzing. It runs the quantum Fourier transform to within one ULP of float32, agrees with Qiskit to 8×10^{-16} over 5,000 random circuits, and reproduces the depolarising channel's fidelity decay.

The defects are the part we expect to outlive the artifact. The first three shared a structure: the tests never placed weight where a plausible wrong implementation would differ. It would be easy to conclude that behavioural testing cannot reach wrong constant data, and we believed something close to it before measuring. It is not true: the present suite kills every one of the 628 meaningful constant mutants we generated. What is true is narrower and more actionable — changing nothing but the states the test programs start from lowers the detection rate to 75.8%, while the static audit is untouched. The thirty defects of the second round say the same about the input space as a whole: every one sat where the hand-written tests never went, and a generator that does not share its author's sense of a typical input re-found most of them in minutes.

So the recommendation is not that testing is hopeless but that three cheap things are worth as much as coverage for numerical code, and especially for numerical code in a language with no type system to lean on: audit the constants directly, where no test input is involved at all; make sure some test places nonzero weight at every index where the data could plausibly be wrong; and let something other than the author choose the inputs. None of the three is a property a coverage number reports. And measure the measuring tools: two of the defects we found this time were in ours, and they had shaped a conclusion in an earlier version of this paper.

Availability

Source, documentation, the full test suite and the measurement scripts are at `https://github.com/shriii1389/quasm` under the MIT licence. Every result in Section 8 is reproducible with the scripts in `tools/`: the cross-validation corpora are regenerable byte for byte from their seeds (`cross_validate.py --seed 1 --n 5000`), the property suite is `properties.py`, the fuzzing runs are `fuzz.py` with their seeds, and the mutation experiments are `mutate.py --full --all-code`, with `--zero-init-rewrite` and `--zero-init-only` for the two arms of Table 2. Programs run through one long-lived RARS process per worker, checked byte for byte against the RARS command line, which brings the whole suite to about a minute. We report these mechanics in the detail that repeatability as a core value asks for [18]. The harnesses ship deliberate-corruption self-tests (`--self-test`), so a reviewer can confirm the checks fail when they should.

References

[1] Shaukat Ali, Paolo Arcaini, Andrea Bombarda, and Tao Yue. Assessing the effectiveness of input and output coverage criteria for testing quantum programs. In *Proceedings of the 14th IEEE Conference on Software Testing, Verification and Validation (ICST)*, 2021.

[2] Benjamin Bichsel, Maximilian Baader, Timon Gehr, and Martin Vechev. Silq: A high-level quantum language with safe uncomputation and intuitive semantics. In *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pages 286–300, 2020.

[3] Tsong Yueh Chen, Fei-Ching Kuo, Huai Liu, Pak-Lok Poon, Dave Towey, T. H. Tse, and Zhi Quan Zhou. Metamorphic testing: A review of challenges and opportunities. *ACM Computing Surveys*, 51(1), 2018.

[4] Cirq Developers. Cirq, 2023.

[5] Koen Claessen and John Hughes. QuickCheck: A lightweight tool for random testing of Haskell programs. In *Proceedings of the Fifth ACM SIGPLAN International Conference on Functional Programming (ICFP)*, pages 268–279, 2000.

[6] Andrew Cross, Ali Javadi-Abhari, Thomas Alexander, Niel De Beaudrap, Lev S. Bishop, Steven Heidel, Colm A. Ryan, Prasahnt Sivarajah, John Smolin, Jay M. Gambetta, and Blake R. Johnson. OpenQASM 3: A broader and deeper quantum assembly language. *ACM Transactions on Quantum Computing*, 3(3), 2022.

[7] Andrew W. Cross, Lev S. Bishop, John A. Smolin, and Jay M. Gambetta. Open quantum assembly language, 2017. arXiv:1707.03429.

[8] Richard A. DeMillo, Richard J. Lipton, and Frederick G. Sayward. Hints on test data selection: Help for the practicing programmer. *Computer*, 11(4):34–41, 1978.

[9] X. Fu, M. A. Rol, C. C. Bultink, J. van Someren, N. Khammassi, I. Ashraf, R. F. L. Vermeulen, J. C. de Sterke, W. J. Vlothuizen, R. N. Schouten, C. G. Almudever, L. DiCarlo, and K. Bertels. eQASM: An executable quantum instruction set architecture. In *Proceedings of the IEEE International Symposium on High Performance Computer Architecture (HPCA)*, 2019. arXiv:1808.02449.

[10] Alexander S. Green, Peter LeFanu Lumsdaine, Neil J. Ross, Peter Selinger, and Benoît Valiron. Quipper: A scalable quantum programming language. In *Proceedings of the 34th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pages 333–342, 2013.

[11] Gian Giacomo Guerreschi, Justin Hogaboam, Fabio Baruffa, and Nicolas P. D. Sawaya. Intel quantum simulator: A cloud-ready high-performance simulator of quantum circuits. *Quantum Science and Technology*, 5(3), 2020.

[12] Daniel Hook and Diane Kelly. Testing for trustworthiness in scientific software. In *Proceedings of the ICSE Workshop on Software Engineering for Computational Science and Engineering (SECSE)*, 2009.

[13] Laura Inozemtseva and Reid Holmes. Coverage is not strongly correlated with test suite effectiveness. In *Proceedings of the 36th International Conference on Software Engineering (ICSE)*, pages 435–445, 2014.

[14] Ali JavadiAbhari, Shruti Patil, Daniel Kudrow, Jeff Heckey, Alexey Lvov, Frederic T. Chong, and Margaret Martonosi. ScaffCC: A framework for compilation and analysis of quantum computing programs. In *Proceedings of the 11th ACM Conference on Computing Frontiers (CF)*, 2014.

[15] Yue Jia and Mark Harman. An analysis and survey of the development of mutation testing. *IEEE Transactions on Software Engineering*, 37(5):649–678, 2011.

[16] Tyson Jones, Anna Brown, Ian Bush, and Simon C. Benjamin. QuEST and high performance simulation of quantum computers. *Scientific Reports*, 9:10736, 2019.

[17] Upulee Kanewala and James M. Bieman. Testing scientific software: A systematic literature review. *Information and Software Technology*, 56(10):1219–1232, 2014.

[18] Shriram Krishnamurthi and Jan Vitek. The real software crisis: Repeatability as a core value. *Communications of the ACM*, 58(3):34–36, 2015.

[19] Benjamin Landers. RARS — RISC-V assembler and runtime simulator. `https://github.com/TheThirdOne/rars`.

[20] Vu Le, Mehrdad Afshari, and Zhendong Su. Compiler validation via equivalence modulo inputs. In *Proceedings of the 35th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pages 216–226, 2014.

[21] Jiacheng Liu, Yuhao Lee, Hanbo Deng, Christopher Clayton, Guanhua Yang, and Xin Wu. RISC-Q: A generator for real-time quantum control system-on-chips compatible with RISC-V, 2025. arXiv:2505.14902.

[22] William M. McKeeman. Differential testing for software. *Digital Technical Journal*, 10(1):100–107, 1998.

[23] Eñaut Mendiluze, Shaukat Ali, Paolo Arcaini, and Tao Yue. Muskit: A mutation analysis tool for quantum software testing. In *Proceedings of the 36th IEEE/ACM International Conference on Automated Software Engineering (ASE), Tool Demonstrations*, 2021.

[24] Andriy Miranskyy, Lei Zhang, and Javad Doliskani. Is your quantum program bug-free? In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering: New Ideas and Emerging Results (ICSE-NIER)*, 2020.

[25] Mariia Mykhailova and Krysta M. Svore. Teaching quantum computing through a practical software-driven approach: Experience report. In *Proceedings of the 51st ACM Technical Symposium on Computer Science Education (SIGCSE)*, pages 1019–1025, 2020.

[26] Michael A. Nielsen and Isaac L. Chuang. *Quantum Computation and Quantum Information*. Cambridge University Press, 10th anniversary edition, 2010.

[27] Matteo Paltenghi and Michael Pradel. Bugs in quantum computing platforms: An empirical study. *Proceedings of the ACM on Programming Languages*, 6(OOPSLA1), 2022. Article 86.

[28] Matteo Paltenghi and Michael Pradel. MorphQ: Metamorphic testing of the Qiskit quantum computing platform. In *Proceedings of the 45th IEEE/ACM International Conference on Software Engineering (ICSE)*, 2023.

[29] Qiskit contributors. Qiskit: An open-source framework for quantum computing, 2023.

[30] Quantum AI team and collaborators. qsim, 2020.

[31] Stefan Seegerer, Tilman Michaeli, and Ralf Romeike. Quantum computing as a topic in computer science education. In *Proceedings of the 16th Workshop in Primary and Secondary Computing Education (WiPSCE)*, 2021.

[32] Sergio Segura, Gordon Fraser, Ana B. Sanchez, and Antonio Ruiz-Cortés. A survey on metamorphic testing. *IEEE Transactions on Software Engineering*, 42(9):805–824, 2016.

[33] Damian S. Steiger, Thomas Häner, and Matthias Troyer. ProjectQ: An open source software framework for quantum computing. *Quantum*, 2:49, 2018.

[34] Krysta Svore, Alan Geller, Matthias Troyer, John Azariah, Christopher Granade, Bettina Heim, Vadym Kliuchnikov, Mariia Mykhailova, Andres Paz, and Martin Roetteler. Q#: Enabling scalable quantum computing and development with a high-level DSL. In *Proceedings of the Real World Domain Specific Languages Workshop (RWDSL)*, 2018.

[35] Jeffrey M. Voas. PIE: A dynamic failure-based technique. *IEEE Transactions on Software Engineering*, 18(8):717–727, 1992.

[36] Kenneth Vollmar and Pete Sanderson. MARS: An education-oriented MIPS assembly language simulator. In *Proceedings of the 37th SIGCSE Technical Symposium on Computer Science Education*, pages 239–243, 2006.

[37] Jiyuan Wang, Qian Zhang, Guoqing Harry Xu, and Miryung Kim. QDiff: Differential testing of quantum software stacks. In *Proceedings of the 36th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pages 692–704, 2021.

[38] Xuejun Yang, Yang Chen, Eric Eide, and John Regehr. Finding and understanding bugs in C compilers. In *Proceedings of the 32nd ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pages 283–294, 2011.